SpoutPlugin

Unreal Engine 5 Spout Sender / Receiver Plugin

User Manual

Plugin version	1.0.0
Engine target	Unreal Engine 5.3 - 5.8
Module	SpoutPlugin (Runtime)
Platform	Windows 64-bit, DirectX 12 RHI
Documentation date	2026-09-12

GPU texture sharing between Unreal Engine and external real-time applications such as TouchDesigner, bridging Direct3D 12 and Spout's Direct3D 11 shared textures.

Table of Contents

About This Manual 3

1. What SpoutPlugin Is 3

2. Requirements 3

3. Installation and Project Setup 4

- Precompiled binary . . . 4
- Source build . . . 4
- Set DirectX 12 . . . 4
- Enable the plugin . . . 4
- Verify it loaded . . . 4

4. Quick Start: Sending the Game Viewport 5

5. Quick Start: Sending a Render Target 5

6. Quick Start: Receiving a Spout Stream 6

7. Sender Workflow 6

8. Receiver Workflow 7

9. SceneCapture2D and Render Target Notes 8

10. Materials 9

11. Blueprint API Reference 9

12. Example Blueprint Setups 13

13. Packaging and Distribution 13

14. Performance Notes 14

15. Limitations 14

16. Troubleshooting 15

17. Best Practices 16

18. Appendix 16

About This Manual

This manual explains how to use SpoutPlugin in Unreal Engine 5, mainly through Blueprints. It is written for Unreal users and technical artists. You do not need to write C++ to use the plugin. Short function signatures appear only in the Blueprint API reference, as the source of truth for each node's inputs and outputs.

Follow the text steps in this guide. Optional editor illustrations are included when available; maintainer capture instructions are kept separately in Docs/SCREENSHOT_CHECKLIST.md.

1. What SpoutPlugin Is

Spout is a Windows technology for sharing GPU textures between real-time applications with no disk or network transfer. One application publishes a texture (a **sender**) and another reads it live (a **receiver**). It is widely used to move video between tools such as TouchDesigner, Resolume, Notch, and OBS.

SpoutPlugin brings Spout into Unreal Engine 5. It sends Unreal's rendered output, or any render target, to other Spout applications, and receives Spout streams back into Unreal as textures and materials. The default transfer path stays on the GPU; CPU receiver fallback uses system memory for readback and upload.

The technical problem it solves: Spout shares DirectX 11 textures, but modern UE5 runs on the DirectX 12 RHI. The plugin bridges UE's D3D12 textures to Spout's D3D11 shared textures on the GPU. By default (the shared-slot path) it copies through plugin-owned slot textures with its own native D3D11 device and never wraps UE's textures; the earlier D3D11-on-D3D12 path remains selectable with `r.Spout.SharedSlotInterop 0` (see Limitations). You do not interact with any of this directly - it is why DirectX 12 is required and why the plugin is Windows-only.

Typical use cases

- Send an Unreal scene or camera into TouchDesigner for compositing, projection mapping, or LED output.
- Receive a live feed from another tool into Unreal and map it onto a mesh or use it in a material.
- Build a real-time pipeline between Unreal and a media-server or VJ setup.

What it does and does not do

Does: GPU texture send and receive over Spout, from the game viewport or a render target, into a texture, a dynamic material instance, or an optional render target.

Does not: it is not a video codec, not a network streaming solution, and not a color-management system. It does not apply gamma or color correction on send. It is Windows-only and requires DirectX 12.

2. Requirements

- Windows 64-bit.
- Unreal Engine 5 (5.3 through 5.8).
- DirectX 12 RHI enabled. The plugin creates its D3D11 interop devices on UE's D3D12 device and will not initialize on any other RHI; it logs `SpoutPlugin requires DX12 RHI.` and stops.
- `Spout.dll`, bundled under `ThirdParty/Spout/lib/amd64/` and staged to the plugin's `Binaries/Win64/` at build time.

- For source builds and for packaging a game (the supported workflow): Visual Studio 2022 with the C++ workload and toolchain and a Windows SDK supported by your engine version. Opening a project with the prebuilt plugin in a matching editor does not need a compiler.
- Install into your project's `Plugins/` folder, the qualified consumer layout. Qualify engine-wide installations separately.
- Blueprint-only project with the plugin in its `Plugins/` folder: in the UE 5.8 test the packaged game did not contain the plugin and reported that module `SpoutPlugin` could not be found. The verified workaround is to add a C++ class to the project before packaging.

Both applications sharing a stream must run on the same GPU. Spout's shared-texture model does not span GPUs.

3. Installation and Project Setup

Precompiled binary

1. Download the ZIP matching your exact engine version (prebuilt binaries are on Patreon).
2. Extract the `SpoutPlugin` folder into `MyProject/Plugins/`.
3. Confirm `Spout.dll` is at `Plugins/SpoutPlugin/Binaries/Win64/Spout.dll`.
4. Set the project RHI to DirectX 12 (below).
5. Open the project. If Unreal offers to rebuild because of an engine-version mismatch, use the correct ZIP or build from source.

Source build

1. Use a C++ project, the supported source-build workflow (add a C++ class to a Blueprint-only project first), and copy `SpoutPlugin` into `MyProject/Plugins/`.
2. Set the project RHI to DirectX 12.
3. Generate Visual Studio project files from the `.uproject`.
4. Build from Visual Studio, or let the editor compile on open.

`Spout.dll` is staged automatically to the plugin's `Binaries/Win64` via Unreal's runtime dependency system. Do not copy it by hand.

Set DirectX 12

Edit > Project Settings > Platforms > Windows > Default RHI > DirectX 12, then restart the editor.

Enable the plugin

A plugin in the project's `Plugins/` folder is enabled by default. Confirm in Edit > Plugins by searching "Spout Plugin". Restart the editor if you change the enabled state.

Verify it loaded

Open the Output Log (Window > Output Log) and look for `Spout Plugin Loaded` and, after first use of any node, `SpoutDX11on12 Initialized.`. If `Spout.dll` is missing, the plugin logs an explicit startup error naming the paths it checked.

4. Quick Start: Sending the Game Viewport

This is the fastest way to get Unreal's rendered image into another application. It sends the game window backbuffer - exactly what you see in Play mode.

Build the Blueprint

1. Create a Blueprint Actor (or open the Level Blueprint) and open its Event Graph.
2. Right-click the canvas, search "Spout Sender", and place the node.
3. Connect Event Tick to the Spout Sender execution input. Tick calls it every frame, which requests continuous frame updates; registration persists until close.
4. Set **Spout Name** to a unique, stable name, for example `UnrealViewport`. The receiver must use this exact name.
5. Set **Send Texture From** to `GameViewport`.
6. Leave **Texture Render Target 2D** empty - it is ignored in this mode.
7. If the actor is a Blueprint Actor, place one instance in the level so its graph runs.

Receive it in another application

Start your external Spout receiver (for example a Spout In TOP in TouchDesigner), then choose `UnrealViewport` from its sender list. You should see the live Unreal image.

> The viewport is captured when the window is ready to present, so the game window must be actively rendering and not minimized. If nothing appears, confirm Play is running and the name matches exactly.

Optional: send on a timer instead of every frame

If you do not need full frame-rate output, drive the sender from a looping timer instead of Tick. This reduces GPU copy cost. Put the Spout Sender call in a function or custom event and trigger it with Set Timer by Event.

5. Quick Start: Sending a Render Target

Sending a render target gives you control over exactly what is shared - a specific camera, a clean feed without UI, or a custom resolution. The usual way to fill a render target is with a SceneCapture2D actor.

Set up the capture

1. Create a `TextureRenderTarget2D` asset. RGBA8 sRGB is the recommended format. You can also create one at runtime with the Create Texture Render Target 2D node.
2. Add a `SceneCapture2D` actor to the level and aim it at what you want to send.
3. In the SceneCapture2D Details, set **Texture Target** to the render target from step 1.
4. Set **Capture Source** to **Final Color (HDR) in Linear Working Color Space**. Qualify other capture sources with known pixels and matching receiver color space.

Send the render target

5. On Event Tick or a timer, call **Spout Sender**.
6. Set **Send Texture From** to `TextureRenderTarget2D`.
7. Set **Texture Render Target 2D** to the same render target asset.
8. Set a **Spout Name** and view the stream in your external application.

If the output is black, the cause is almost always the Capture Source in step 4. See section 9 for the full black-output checklist.

6. Quick Start: Receiving a Spout Stream

Receiving pulls a stream from any Spout application into Unreal. You get a live texture, and optionally a ready-to-use material instance and a render target copy.

Build the Blueprint

1. Create a Dynamic Material Instance once and store it in a variable. On Tick or a timer, call **Spout Receiver With Material**.
2. Set **Spout Name** to exactly match the sender, including case.
3. Pass the stored MID to **Consumer Material** and set **Texture Parameter Name**. Leave the material empty for texture-only use.
4. Optionally expand the advanced pins and set **Optional Output Render Target** to copy the frame into a render target as well.
5. Read **Out Texture** and the return **bool**. True means accepted and queued, not that pixels have finished copying.
6. Branch on the return value and validate Out Texture before use; the sender may not be running yet.
7. Apply your stored MID to a mesh with Set Material, or sample Out Texture directly.

If the receiver returns false or the image is blank, check the name match first, then confirm the sender is actually running and being called continuously.

7. Sender Workflow

The sender publishes a UE texture to other Spout applications as a shared GPU texture. Two sources are available, chosen by the Send Texture From pin.

GameViewport

Sends the active window backbuffer - what you see in Play mode. The plugin hooks the point in the frame where the window is ready to present, so the capture is correctly timed. The window must be rendering and not minimized. Leave the Texture Render Target 2D pin empty.

TextureRenderTarget2D

Sends a render target you provide. Drive it with SceneCapture2D, Draw Material to Render Target, or another render-to-texture method. The request is accepted on the game thread; its current graphics resource is resolved and copied on the render thread.

Stream name rules

- The name must match the receiver exactly, including case.
- Only one sender can own a name at a time. Two senders sharing a name conflict.
- Sender and receiver entries are tracked separately, so a sender and a receiver in the same project may use the same name without interfering.

Lifetime

Call the sender on Tick or a timer to publish changing images. The last image remains advertised while the sender is paused. The shared texture is created and resized when the source size or format changes. Call **Close Sender** to stop advertising; it affects only the sender entry, leaving any receiver with the same name open.

Performance

- Cost scales with resolution. Use a timer when full frame rate is not needed.
- By default UE copies the source into a plugin-owned slot texture (the render thread waits at most 1 s for that copy) and the plugin's native D3D11 device copies the slot into the Spout shared texture; with `r.Spout.SharedSlotInterop 0` the source is wrapped through a D3D11-on-D3D12 device instead.
- The shared texture and the slot (or wrap) are cached and reused while size and format are stable.
- No CPU readback happens on the send path - it is a GPU-to-GPU copy.

Colour space and access

- No sender-side colour correction is applied: the pixel bytes and alpha are copied unchanged. Every 8-bit source is published as plain _UNORM, never _UNORM_SRGB, because Spout 2.007's OpenGL receivers (such as the Spout 2.007 Demo Receiver) refuse B8G8R8A8_UNORM_SRGB.
- Keeping the bytes unchanged does not keep the sRGB format tag: an sRGB render target (Force Linear Gamma off) and a Force Linear Gamma render target both publish _UNORM, so a receiving application cannot tell the source's colour intent from the format. _UNORM does not mean the bytes are linear: by the Spout convention 8-bit _UNORM bytes are gamma-encoded (sRGB), which is what an sRGB render target produces and what receivers on Auto expect. A Force Linear Gamma target stores linear values, so set its receiver's Color Space to Linear.
- Each publish takes Spout's cross-process access lock. If a receiver holds the texture, the publish waits at most r.Spout.AccessWaitMs (default 2 ms) for it; if it is still busy, that frame is skipped, the next one is published once the texture is free, and a throttled "shared-texture access was unavailable" warning is logged.

8. Receiver Workflow

The receiver consumes a stream and exposes it to Unreal. Call Spout Receiver continuously with a matching name.

Output	Type	Notes
Out Texture	UTexture2D (by ref)	Transient texture, resized to the stream, updated each call.
Out Mat	UMaterialInstanceDynamic (by ref)	Created from Input Material on the first call, with Out Texture bound to the parameter.
Optional Output Render Target	UTextureRenderTarget2D (input)	If provided, the frame is also copied into it; resized automatically.

Finding senders: use the **Spout Info** node to list active sender names, widths, and heights - useful for a debug overlay or a dropdown. Names must match exactly. The receiver reconnects automatically when the sender appears or changes size.

Color Space (advanced pin) decides how the received bytes are read; see the Blueprint API Reference. An 8-bit _UNORM format does not say whether the bytes are sRGB-encoded or linear (this plugin's sender publishes _UNORM for sRGB and Force Linear Gamma render targets alike), so choose Linear yourself

for a linear sender.

Access: each copy takes Spout's cross-process access lock. If a peer (the sender or another receiver) holds the texture, the copy waits at most r.Spout.AccessWaitMs (default 2 ms) for it; if it is still busy, the receiver tries again on a later call and a throttled "shared-texture access was unavailable" warning is logged. That call copies the sender's newest frame, so a frame the sender replaced in the meantime is lost, not queued.

Sampling: the receiver copies at most once per Unreal frame and gets the sender's newest frame; Spout does not queue frames. When the sender and Unreal run at the same frame rate, their timing drifts in and out of alignment, and near alignment single frames are skipped and the previous one is shown for an extra frame, without any access contention. A higher Unreal frame rate reduces this but does not guarantee every frame. See Limitations.

Receive paths (CVar r.Spout.GPUReceiver)

The receiver has two implementations, selected at runtime by a console variable. Both produce identical images; the difference is performance only.

Value	Path	Behavior
1 (default)	GPU-direct	Shared texture copied into the UE destination on the GPU: by default through a plugin-owned slot (the render thread waits at most 100 ms for the native copy), via D3D11-on-12 with r.Spout.SharedSlotInterop 0. No staging texture, no CPU copy.
0	Legacy CPU readback	Copies to a staging texture, maps it, uploads via RHIUpdateTexture2D. For debugging; more expensive at high resolution.

Set it in the console, in `DefaultEngine.ini` under `[SystemSettings]`, with a `-ExecCmds` launch argument, or from a Blueprint Execute Console Command. It is read once per receive call, so changes apply on the next call with no restart.

The `OutputRenderTarget` field on FSenderStruct is unused. The functional render-target output is the Optional Output Render Target pin on the Spout Receiver node.

9. SceneCapture2D and Render Target Notes

Recommended Capture Source: Final Color (HDR) in Linear Working Color Space.

Other capture sources: SceneColor HDR requires qualification of capture settings, exposure, format and receiver interpretation. The plugin does not tone-map or convert pixels.

Recommended render target format: RGBA8 sRGB (start here). RGB10A2 is also tested and useful to reduce banding.

The plugin shares the texture as-is and does not color-correct on send, so color correctness depends on the Capture Source and render target format. The tested combination is Final Color (HDR) in Linear Working Color Space with an RGBA8 sRGB render target.

Black-output checklist

- Capture Source is Final Color (HDR) in Linear Working Color Space.
- The render target is assigned to the SceneCapture2D Texture Target.
- The capture is active and pointed at visible geometry.
- You are sending the same render target asset you assigned to the capture.
- The render target uses a tested format.

10. Materials

The receiver can hand you a ready-to-use material instance with the live texture already bound. To use it, your material needs a Texture Parameter whose name matches what you pass to the node.

1. Create a material with a **Texture Parameter** (the example content is M_SpoutMaterial). Connect it to Base Color or Emissive.
2. Name the parameter. If you leave Texture Parameter Name empty on the node, the plugin uses `SpoutTexture`, so naming the parameter `SpoutTexture` matches the default.
3. Create a Dynamic Material Instance from the base material once, and retain it in a Blueprint variable.
4. Supply that MID to **Spout Receiver With Material** each tick. The plugin binds the shared received texture without allocating another material.
5. Use Set Material with your MID. Release Receiver Material Binding detaches just that parameter, preserving later edits; Close Receiver closes the shared session.

If you do not need a material, leave Input Material empty and use Out Texture directly, or use the Optional Output Render Target.

11. Blueprint API Reference

All nodes are static functions on `USpoutBPFunctionLibrary`, category `Spout`. The function signatures below are the source of truth; Blueprint pin names match the parameter names.

Node	Returns
Spout Sender	bool
Close Sender	void
Spout Receiver	bool
Close Receiver	void
Spout Receiver With Material	bool
Release Receiver Material Binding	bool
Get Spout Stream Status	FSpoutStreamStatus
Get Spout Runtime Status	FSpoutRuntimeStatus
Spout Info	bool
Get Sender Info	bool
Set Max Senders	int32
Get Max Senders	void
Create Texture Render Target 2D	UTextureRenderTarget2D*

Spout Sender

```
bool SpoutSender(FName SpoutName, ESpoutSendTextureFrom SendTextureFrom,
UTextureRenderTarget2D* TextureRenderTarget2D)
```

Publishes a texture to Spout; the GPU copy runs on the render thread. Returns true when accepted or queued. Provide the render target when using the TextureRenderTarget2D source. Call continuously. Common mistakes: empty render target with the render-target source; calling once; name mismatch.

Close Sender

```
void CloseSender(FName SpoutName)
```

Removes the named sender and releases its handle. Scoped to senders only, so a receiver using the same name is unaffected.

Spout Receiver

```
bool SpoutReceiver(const FName SpoutName, UMaterialInterface* InputMaterial,
FName TextureParameterName, UMaterialInstanceDynamic*& OutMat,
UTexture2D*& OutTexture,
UTextureRenderTarget2D* OptionalOutputRenderTarget = nullptr,
ESpoutColorSpace ColorSpace = ESpoutColorSpace::Auto,
bool bAsyncReceive = false)
```

Receives a stream. OptionalOutputRenderTarget and Color Space are advanced-display pins. Returns true when the sender exists and a receive was enqueued. Outputs Out Mat and Out Texture, which are OUTPUTS and not ownership: the received texture belongs to the plugin, which keeps it alive while the receiver is open. Keep the outputs for Close Receiver. C++ can reuse OutMat by reference; Blueprint callers should use the explicit Consumer Material input of Spout Receiver With Material. Texture Parameter Name defaults to SpoutTexture when empty. Upstream frames are consumed once; optional targets catch up separately, including paused senders and recreated targets.

Optional Output Render Target. When provided, the received frame is also copied into it. Its pixel format must match the incoming frame's format exactly and it must be single-sampled (no MSAA); no implicit format conversion is performed. A mismatched target is left completely untouched and the mismatch is logged - the call still succeeds and Out Texture still updates - which is what a target that stays black is telling you. Size differences are not a mismatch: the target is resized to match the stream.

Color Space. Decides how the received bytes are interpreted. **Auto** (the default) follows the format convention the Spout ecosystem publishes: 8-bit frames decode as sRGB, while 10-bit and 16-bit float frames stay linear. **Linear** applies no decoding; use it when the sender publishes linear or data values in an 8-bit format, such as a render target created with Force Linear Gamma, which publishes the same _UNORM format as an sRGB one. **sRGB** forces decoding, and is only representable for 8-bit frames: requesting it for a 10-bit or float frame fails the call and logs, rather than being silently ignored.

Async Receive (Beta). Off by default; an advanced input of both receive nodes and a checkbox under Spout > Experimental on the receiver example. It moves receive work off the render thread, which may improve delivery under load but can increase frame-time spikes or image latency. It applies per sender name: the first accepted receive decides the mode of the shared session, a call asking for the other mode is refused and logged until Close Receiver, and nothing switches an open session. Senders other than BGRA8 or RGBA8, or a worker failure, make the session run synchronously; the stream status reports the requested mode, the effective mode and why. r.Spout.AsyncReceive is retired and ignored.

Spout Receiver With Material

```
bool SpoutReceiverWithMaterial(FName SpoutName,
UMaterialInstanceDynamic* ConsumerMaterial, FName TextureParameterName,
UTexture2D*& OutTexture, UTextureRenderTarget2D* OptionalOutputRenderTarget,
ESpoutColorSpace ColorSpace, bool bAsyncReceive)
```

Use a stored, caller-owned MID input; this node allocates no material. Null supports texture-only use. Optional targets catch up independently while the sender is paused. Consumers sharing a name must agree on resolved color space; close the session before changing interpretation, or let a source pixel-format change establish a new interpretation.

Release Receiver Material Binding

```
bool ReleaseReceiverMaterialBinding(FName SpoutName,
UMaterialInstanceDynamic* ConsumerMaterial, FName TextureParameterName)
```

Releases one parameter without closing the receiver. Restores the previous texture or engine default only while the parameter still points to this receiver image. Later manual edits and other sender bindings are preserved.

Close Receiver

```
void CloseReceiver(FName SpoutName, UTexture2D*& ReceivedTexture,
UMaterialInstanceDynamic*& ReceivedMaterial)
```

Closes the receiver session for that name. There is one session per sender name, so this closes it for every graph receiving it, and it restores material parameters still owned by the session, preserving later changes. The NAME decides what is released, not the pointers: pass back the Out Texture and Out Mat this plugin gave you and both are cleared; pass anything else and it is left untouched with a warning. This node will not destroy a texture it did not create.

Spout Info

```
bool SpoutInfo(TArray<FSenderStruct>& Senders)
```

Fills an array with one entry per active sender. Only Name, Width, Height, and bIsAlive are populated. Returns true if the registry was queried.

Get Sender Info

```
bool GetSenderInfo(FName SpoutName, FSenderStruct& OutSenderStruct)
```

Looks up one sender by name; fills Name, Width, Height, bIsAlive. Returns true if it exists.

Set Max Senders

```
int32 SetMaxSenders(int32 Max)
```

Sets the maximum sender count in Spout's registry and returns the limit actually in effect. Returns 0 when the request could not be applied, either because the requested value was outside the accepted range (1-1024) or because Spout is unavailable. It does not echo the requested value back.

Get Max Senders

```
void GetMaxSenders(int32& Max)
```

Reads the current maximum sender count into Max.

Create Texture Render Target 2D

```
UTextureRenderTarget2D* CreateTextureRenderTarget2D(int32 Width = 1024, int32 Height = 768,
EPixelFormat Format = PF_B8G8R8A8, bool bForceLinearGamma = true)
```

Creates a render target at runtime, sets wrap addressing on both axes, and initializes its resource. The result is NOT rooted: keep a reference to it exactly as you would with Create Render Target 2D, or it will be collected. Returns null for invalid dimensions or for a pixel format the plugin cannot share (supported: B8G8R8A8, R8G8B8A8, A2B10G10R10, FloatRGBA). Force Linear Gamma decides what an 8-bit target's bytes contain; the sender publishes plain _UNORM either way. True (default) stores linear values: set the receiver's Color Space to Linear. False stores sRGB-encoded bytes, the Spout convention, which a receiver on Auto reads correctly.

Get Spout Stream Status

```
FSpoutStreamStatus GetSpoutStreamStatus(FName SpoutName, ESpoutType Type)
```

Returns a game-thread snapshot of the current role's activity, queue admissions/rejections, submitted frames, access misses and native failures. Active can mean awaiting a first frame. Submission does not acknowledge display by another application. Viewport coalescing is outside the non-viewport queue counters; closing a role retires its counters.

Get Spout Runtime Status

```
FSpoutRuntimeStatus GetSpoutRuntimeStatus()
```

Returns initialized interop availability, active roles, pending/peak commands, role/queue-cap refusals and estimated image reservations. Allocation refusals have a separate counter. The query does not initialize the device. See Operational limits and status in the appendix for the default budgets.

Enums

ESpoutType: Sender, Receiver. **ESpoutSendTextureFrom:** GameViewport, TextureRenderTarget2D.

ESpoutColorSpace: Auto (follow the format convention - 8-bit decodes as sRGB, 10-bit and float stay linear), Linear (no decoding), SRGB (force sRGB decoding; representable for 8-bit formats only, and refused otherwise).

Supported formats. The plugin shares B8G8R8A8, R8G8B8A8, A2B10G10R10 and FloatRGBA, including their typeless variants. Anything else is refused: a warning naming the advertised format is written to the log and nothing is published or received for that frame. Unsupported formats are never reinterpreted as B8G8R8A8 and no fallback conversion is fabricated, because advertising a layout the source does not have produces silently garbled frames rather than an honest failure.

Struct: FSenderStruct

Field	Type	Filled by plugin?
Name	FName	Yes (Info nodes)
bIsAlive	bool	Yes (Info nodes)
SpoutType	ESpoutType	No
Width	int32	Yes (Info nodes)
Height	int32	Yes (Info nodes)
TextureColor	UTexture2D*	No
MaterialInstanceColor	UMaterialInstanceDynamic*	No
UserBaseMaterial	UMaterialInterface*	No
UserTextureParameter	FName (default SpoutTexture)	No
OutputRenderTarget	UTextureRenderTarget2D* (unused)	No

Rely only on Name, Width, Height, and bIsAlive from the Info nodes. The remaining fields are available for your own Blueprint use but are not driven by the plugin.

12. Example Blueprint Setups

The plugin ships example content under its `Content/` folder. These are the actual asset names; open them to see working graphs rather than building from scratch.

Asset	Purpose
BP_SpoutSenderExample_	Example sender setup.
BP_SpoutReceiverExample	Example receiver setup.
M_SpoutMaterial	Material with a texture parameter for receiver output.
RT_SpoutSenderTest_	Example render target for sending.
RT_SpoutReceiverTest	Example render target for receiving.

Enable 'Show Plugin Content' in the Content Browser settings to see them.

13. Packaging and Distribution

`Spout.dll` ends up in a different place depending on how the plugin is being used, and conflating them is a common source of "module could not be loaded" reports. In an editor or other modular build, RuntimeDependencies stages it to the plugin's own `Binaries/Win64`. In a packaged game the plugin module is linked into the game executable, so the same `$(BinaryOutputDir)` target resolves to the PROJECT's `Binaries/Win64` beside the .exe, and the plugin's own Binaries folder is not staged at all. Raw `RunUAT BuildPlugin -Package` output does NOT receive the DLL from RuntimeDependencies; it stays under `ThirdParty/Spout/lib/amd64`, which is why `build.ps1` copies it into the release archive explicitly. The DLL is delay-loaded, and the module loads it by full path at startup so the delay-load thunks bind by name.

A release folder contains Binaries (with `Spout.dll`), Content, Resources, Source, and `SpoutPlugin.uplugin`. Precompiled releases must include `Spout.dll` in `Binaries/Win64`.

Releases are built per engine version with `RunUAT BuildPlugin ... -TargetPlatforms=Win64 -Rocket`. Debug symbols are removed, but `Intermediate/Build` is deliberately KEPT: it carries the precompiled UnrealGame Development and Shipping objects and the `.precompiled` manifests that BuildPlugin produces. They are not a supported way to package a game without Visual Studio. Each archive is then reopened and validated before it is published. The repository `build.ps1` does this for UE 5.3 through UE 5.8. A binary built for one engine version will not load in another.

Clean rebuild after an engine upgrade

```
Project/Binaries
Project/Intermediate
Plugins/SpoutPlugin/Binaries
Plugins/SpoutPlugin/Intermediate
```

Delete those, regenerate project files, and rebuild from scratch.

14. Performance Notes

- Cost scales with texture resolution on both send and receive. Start lower and raise as needed.
- Use a timer instead of Tick when you do not need full frame-rate output.
- Leave `r.Spout.GPUReceiver` at its default of 1. The legacy path (0) adds a full-frame copy, map, and upload each frame.
- The send path is a GPU copy through the interop path; there is no CPU readback on send.
- Keep per-frame Blueprint logic around the send and receive calls minimal.
- Watch large textures: a 4K stream at full frame rate is far more expensive than 1080p.

15. Limitations

- Windows 64-bit only.
- DirectX 12 RHI required; the plugin refuses to initialize otherwise.
- Both ends must be on the same GPU.
- Supported formats are BGRA8, RGBA8, RGB10A2 and RGBA16F; color interpretation must match the source.
- No colour conversion on send. Every 8-bit source is published as plain _UNORM with its bytes unchanged; _UNORM_SRGB is never published, because Spout's OpenGL receivers refuse it. See section 7.
- The sRGB format tag is not carried: an sRGB render target and a Force Linear Gamma render target both publish _UNORM, so receivers cannot tell the source's colour intent from the format. _UNORM does not mean linear; by the Spout convention the bytes are sRGB-encoded, and a Force Linear Gamma source needs Color Space = Linear on the receiver.
- Final Color (HDR) in Linear Working Color Space is a starting point; qualify Capture Source, exposure and output format together.
- One sender per name.
- Senders and receivers must be called continuously.
- Receiving samples a stream once per Unreal frame and does not queue frames. At equal frame rates the two clocks drift in and out of alignment; near alignment single frames are skipped and the previous one repeated, with no access contention involved. In 1080p 60 fps testing this lowered delivery to 87 to 89 % of distinct frames in individual 10-second windows, while whole runs delivered about 99 %. r.Spout.AccessWaitMs does not change it; a higher Unreal frame rate reduces it but does not

guarantee every frame, because jitter, hitches, stalls and contention can still lose frames.

- The access wait (r.Spout.AccessWaitMs, default 2 ms) applies to each attempt, not to a frame, so several busy streams add up: in testing, a peer that locked eight 1080p streams added about 13 ms of render-thread waiting per frame. 0 restores zero-wait acquisition only; the plugin's other fixes stay active.
- A keyed-mutex sender that holds its texture for tens of milliseconds can cause runs of consecutive skipped frames, with or without the access wait. With one 1080p 60 fps keyed test sender holding its texture 20 ms on every tenth frame, delivery was 84 % with up to 14 frames skipped in a row. Qualify keyed senders with their actual workload.
- Spout Info and Get Sender Info populate only Name, Width, Height, bIsAlive.
- The descriptor sets IsBetaVersion to false; qualification is still required. Native failures are reported and transient connection failures retry with backoff.

16. Troubleshooting

The plugin logs to the `LogSpoutPlugin` category, so `Project/Saved/Logs/ProjectName.log` is the first place to look for any of these.

Symptom	What to do
Plugin does not compile	Confirm it is in the project Plugins folder, you are on UE 5.3-5.8 with VS 2022; regenerate project files; clean rebuild after engine upgrades.
Module could not be loaded	Binary must match your exact engine; Spout.dll must be in Binaries/Win64. Read the log for the startup error.
Spout.dll missing	Copy it from ThirdParty/Spout/lib/amd64/ to Binaries/Win64/, or rebuild from source.
Sender not visible externally	Confirm DirectX 12; confirm continuous calls; one sender per name; for GameViewport the window must be rendering; look for SpoutDX11on12 Initialized.
Receiver returns invalid output	Exact name match; check the return value; validate outputs; texture parameter name must match the material (default SpoutTexture).
Black texture from SceneCapture2D	Set Capture Source to Final Color (HDR) in Linear Working Color Space.
Wrong colors or gamma	Adjust render target format and Capture Source; try RGBA8 sRGB first, then RGB10A2. 8-bit sources are published as _UNORM whatever their gamma, so set Color Space = Linear on the receiver (Unreal or external) for a Force Linear Gamma source.
Spout Demo Receiver: "No sender detected" for an sRGB render target	Fixed in 1.0.0. Earlier release candidates published 8-bit sRGB render targets as _UNORM_SRGB, which Spout 2.007's OpenGL receivers refuse; the sender now publishes plain _UNORM with unchanged bytes.
Log: "shared-texture access was unavailable"	Another application still held the shared texture after the access wait (r.Spout.AccessWaitMs, default 2 ms). The sender skips that frame and publishes the next; the receiver copies the sender's newest frame on a later call. Throttled to one line per stream every 5 seconds; Get Spout Stream Status counts every miss. The wait applies to each attempt on the render thread, not to a frame, so several locked streams add up; r.Spout.AccessWaitMs 0 never waits (zero-wait acquisition only; other fixes stay active).
Frames skip or repeat, no access misses	Sampling, not contention: the receiver reads once per Unreal frame and a sender at the same frame rate drifts in and out of alignment with it. r.Spout.AccessWaitMs does not change this; a higher Unreal frame rate reduces it. See Limitations.
Stream name mismatch	Use the exact external name from Spout Info. Local FName keys do not distinguish case.
Multiple senders conflict	Unique, stable names per stream.

Symptom	What to do
Performance drops	Keep the GPU receiver default; reduce resolution or update rate; prefer timers.
Crashes or instability	Update GPU drivers; avoid swapping render target resources at runtime; test resolutions; reproduce in a minimal project.
External app cannot see stream	Confirm Spout/format support and that both apps are on the same GPU.
After an engine upgrade	Clean rebuild (delete the four Binaries/Intermediate folders), regenerate, rebuild.

If you can reproduce a missing-DLL failure, keep the startup log line as a diagnostic.

17. Best Practices

- Use stable, unique Spout names and keep sender and receiver names identical.
- Start at a low resolution and raise it only when needed.
- Use a timer when full frame rate is not required.
- Always validate Out Mat and Out Texture before applying them.
- Keep the render target format simple: RGBA8 sRGB first.
- For SceneCapture2D sends, start with Final Color (HDR) in Linear Working Color Space and qualify the chosen capture pipeline.
- Test in a minimal project before integrating into production.
- Record the engine version and the plugin binary version together; binaries are version-specific.

18. Appendix

File structure

```
SpoutPlugin.uplugin
Source/SpoutPlugin/
SpoutPlugin.Build.cs
Public/ SpoutBPFunctionLibrary.h, SpoutModule.h
Private/ SpoutBPFunctionLibrary.cpp, SpoutModule.cpp,
SpoutSender.*, SpoutReceiver.*, SpoutD3DContext.*,
SpoutD3DUtils.*, SpoutSenderRegistry.*, SpoutTextureUtils.*
ThirdParty/Spout/ include/, lib/amd64/ (Spout.dll, Spout.lib)
Content/ example Blueprints, material, render targets
Resources/ Icon128.png
Docs/ documentation and this manual
build.ps1, build.bat
```

Module and dependency summary

Single Runtime module `SpoutPlugin`, Win64 only. Public dependencies: Core, CoreUObject, Engine, RHI, RenderCore. Private: ApplicationCore, Slate, SlateCore, Projects, D3D12RHI, D3D11RHI, plus the engine DX12 and DX11 third-party libraries. Spout.dll is delay-loaded and staged as a runtime dependency.

Operational limits and status

All public APIs require the game thread. Open streams and their last image persist until close. Editor world cleanup closes all plugin streams across PIE worlds; packaged callers must close at their own EndPlay or level boundary. Device removal requires an application restart.

GetSpoutStreamStatus exposes scheduling, submitted-frame, contention and native-failure counters. GetSpoutRuntimeStatus exposes active roles, pending work and retained-image reservations. These

snapshots do not acknowledge delivery by another application.

Defaults: 32 active roles, 128 pending commands, 4 per stream, 32 consumers per receiver, 8192 pixels per dimension, 128 MiB per frame and 1024 MiB aggregate image reservations. Console variables under r.Spout.Max control admissions. Cancelled commands remain counted until drained. Old outputs retained by callers stay charged until GC/resource release. Caller-owned textures and driver/engine allocations are additional.

Named and keyed texture access wait at most r.Spout.AccessWaitMs (default 2 ms, clamped to 0 to 10) for a busy peer, per attempt and on the render thread; 0 never waits (zero-wait acquisition only; the plugin's other fixes stay active). It is not a per-frame bound: several busy streams can each add up to that bound in one frame (about 13 ms per frame with eight locked 1080p streams in testing), and Windows scheduling can make a wait last longer than requested (about 16 ms at the default Windows timer resolution; Unreal normally requests 1 ms). A texture still busy after the wait skips that transfer until a later call. GPU completion can still stall rendering. Receiving samples each stream once per Unreal frame and does not queue frames, so frames can be skipped or repeated without contention when both sides run at the same frame rate. Release tools require PowerShell 7.2 or newer. Candidate qualification installs exact ZIP archives into fresh consumer hosts and runs the shipped examples.

Third-party attribution

- Spout: http://spout.zeal.co/
- Based on the original repository by zuyi53: https://github.com/zuyi53/Spout-UE5

License

MIT License. Commercial use is allowed, but reselling the plugin as-is as a standalone product goes against the spirit of the project.

Glossary

Term	Meaning
Spout	Windows GPU texture-sharing technology between real-time applications.
Sender	An application publishing a shared texture.
Receiver	An application reading a shared texture.
Render Target	A UTextureRenderTarget2D; a texture you can render into and read back.
SceneCapture2D	An Unreal actor that renders a view into a render target.
RHI	Rendering Hardware Interface, Unreal's graphics API abstraction.
D3D12	DirectX 12, the graphics API Unreal uses; required by this plugin.
D3D11-on-D3D12	An interop layer exposing D3D12 resources to D3D11 code; used by the fallback interop path (r.Spout.SharedSlotInterop 0).
Shared slot	A texture the plugin creates on UE's D3D12 device in a shared heap and opens on its native D3D11 device; the default interop path moves pixels through it.
Shared texture	A GPU texture exposed via a shared handle so another process can read it.